

JAVA INTERVIEW PREPARATION

CHAPTER 3

JAVA STRINGS: IMMUTABILITY, POOLING, AND RUNTIME BEHAVIOR

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Java Strings: Immutability, the String Pool, and Runtime Behavior

Senior / Lead Java Developer Reading Chapter

Strings appear in almost every Java application: identifiers, URLs, SQL fragments, JSON properties, log messages, configuration values, and user input. Because they are familiar, it is easy to treat them as simple character containers. In reality, `String` combines value semantics, immutability, compiler support, JVM pooling, Unicode behavior, and performance characteristics.

A senior developer should understand not only how to compare or concatenate strings, but why Java can safely share them, when the pool is involved, what `intern()` actually returns, and how apparently harmless string operations affect allocation and production memory.

A String Is an Immutable Value

`String` is a final class. Once a String object has been constructed, its character sequence cannot be changed through the public API.

```
String name = "Alice";
String upper = name.toUpperCase(Locale.ROOT);

System.out.println(name); // Alice
System.out.println(upper); // ALICE
```

`toUpperCase()` does not modify `name`. It returns a String representing the result. Some methods may return the same object when no change is necessary, but callers must reason from the value contract rather than object identity.

```
String value = "ALICE";
String result = value.toUpperCase(Locale.ROOT);

value.equals(result); // true
value == result;      // implementation may return the same reference
```

The important guarantee is equality of value, not whether a new allocation occurred.

Immutability gives Java several benefits:

- A String can be shared by unrelated callers without defensive copying.
- It is safe to use as a key in hash collections because its value-based hash cannot change.
- The JVM can pool literals and return shared references.
- Security-sensitive APIs can receive a path, class name, or algorithm name without the caller changing that String during validation.
- Concurrent readers require no synchronization to protect the String's contents.

Immutability does not make a variable constant. A variable can be reassigned to another String:

```
String status = "PENDING";
status = "APPROVED";
```

The original `"PENDING"` object was not modified. The variable now refers to another value.

```
Before reassignment
status -----> "PENDING"

After reassignment
status -----> "APPROVED"

"PENDING" remains unchanged
```

This distinction between immutable objects and mutable references is important in concurrent and domain modeling discussions.

String Representation and Unicode

The Java API presents a String as a sequence of UTF-16 code units. A `char` is one 16-bit code unit, not necessarily one complete human-readable character.

Many common characters fit in one code unit. Some Unicode characters, including many emoji and historic scripts, require a surrogate pair.

```
String text = "\u0041\u200B\u200C"; // A followed by one emoji

text.length(); // 3 UTF-16 code units
text.codePointCount(0, text.length()); // 2 Unicode code points
```

Code that truncates, reverses, or indexes user-visible text by `char` can split a surrogate pair. Even code-point counting does not fully represent what a user sees as one character because combining marks and grapheme clusters can combine multiple code points.

For identifiers defined by an ASCII protocol, code-unit operations may be sufficient. For names and natural-language text, use APIs and libraries that match the required Unicode and locale semantics.

Modern JDKs may store suitable strings compactly using a byte array plus an encoding indicator, reducing memory for Latin-1 content. That is an implementation optimization. Application code should depend on the String contract, not its private fields, because internal representation can change across JDK versions.

Literals and the String Pool

A String literal receives special treatment from the Java language and JVM.

```
String first = "Java";
String second = "Java";
```

Both literal expressions refer to the canonical pooled String for the value `"Java"` within the relevant runtime context.

```
first -----+
              |
              +-----> pooled String "Java"
              |
second -----+
```

Canonical means the one shared representative selected for an interned value. There may be other String objects with the same characters, but the pool maintains one canonical reference for that interned value.

Literals are recorded in class metadata and resolved as classes are loaded and linked. In modern JVMs, the interned String objects themselves are managed in ordinary heap memory, while the pool data structure provides canonical lookup.

The pool is possible because String is immutable. If one caller could change the shared "Java" object to "Python", every other caller using that literal would observe corrupted state.

Literals Versus new String(...)

The expression below explicitly requests another String object:

```
String pooled = "Java";
String distinct = new String("Java");

pooled == distinct;    // false
pooled.equals(distinct); // true
```

The references can be pictured as:

```
pooled -----> canonical pooled object "Java"
distinct -----> separate String object "Java"
```

`new String("Java")` always creates the object requested by `new`. The literal refers to the pooled object. If the literal has not previously been resolved in that runtime, resolving it may also establish the pooled representative. This is why interview questions about "how many objects" must specify whether the literal already exists and whether they mean objects associated with the expression or newly created at that moment.

In application code, `new String(existingString)` is normally unnecessary. It does not create a mutable or more secure copy; the result is still immutable. It adds allocation without changing the value contract.

Why == Sometimes Appears to Work

The `==` operator compares references. It does not compare String content.

```
String a = "order";
String b = "order";

a == b; // true because both literals use the pooled reference
```

This can create a misleading test that passes during development.

```
String c = new StringBuilder()
    .append("or")
    .append("der")
    .toString();

a == c;    // false
a.equals(c); // true
```

Business logic must use `equals()` for content.

When the receiver may be null, a known constant can be the receiver:

```
if ("APPROVED".equals(status)) {
    processApprovedOrder();
}
```

Alternatively, `Objects.equals(left, right)` provides null-safe comparison when either value may be null.

```
Objects.equals(first, second);
```

Reference comparison is appropriate only when identity is intentionally being tested, such as verifying canonicalization in controlled infrastructure code. It is not a substitute for value comparison.

What `intern()` Actually Does

`String.intern()` returns the canonical pooled reference for the receiver's value.

```
String generated = new StringBuilder()
    .append("Ja")
    .append("va")
    .toString();

String canonical = generated.intern();

canonical == "Java";      // true
generated.equals(canonical); // true
```

The method does not modify `generated`. It performs a value-based pool lookup:

```
generated "Java"
  |
  | intern()
  v
search String pool for equal value
  |
  |-- found ----> return existing canonical reference
  |
  |-- absent ----> establish and return canonical reference
```

Interning can reduce duplicate objects when an application has an enormous number of repeated values drawn from a controlled, relatively small vocabulary. It can also make identity comparison technically possible after every participant canonicalizes values.

However, interning is not a default application optimization. Pool lookup has a cost, interned values can remain reachable for significant periods, and high-cardinality or attacker-controlled input can increase memory pressure. Ordinary domain code should prefer value equality. Use interning only after measuring duplication and understanding the value lifecycle.

A bounded application-level canonicalization cache may provide clearer ownership, size limits, expiry, and metrics than the global String pool.

Compile-Time and Runtime Concatenation

The compiler can fold constant expressions.

```
String first = "Java" + " Interview";
String second = "Java Interview";

first == second; // true for compile-time constants
```

Both expressions become the same constant value and use its pooled representative.

Concatenation involving runtime values is different:

```
String role = request.role();
String message = "Selected role: " + role;
```

The compiler and runtime generate code to construct the result. Modern Java uses dynamic concatenation strategies that the JVM can optimize, rather than requiring developers to predict one fixed desugaring implementation.

For a small expression, `+` is clear and normally appropriate:

```
String label = customerNumber + " - " + customerName;
```

Repeated concatenation in a loop can create a sequence of intermediate results because each `String` result is immutable.

```
String csv = "";

for (String value : values) {
    csv = csv + value + ",";
}
```

Conceptually, every iteration creates another value containing the previous content plus the addition. As the result grows, earlier content is repeatedly copied.

A mutable builder is the appropriate construction tool:

```
StringBuilder csv = new StringBuilder();

for (String value : values) {
    if (!csv.isEmpty()) {
        csv.append(',');
    }

    csv.append(value);
}

String result = csv.toString();
```

For delimiter-based joining, the higher-level API may be clearer:

```
String result = String.join(",", values);
```

The senior decision is not "always use `StringBuilder`." It is to use the clearest operation and recognize when repeated immutable reconstruction becomes material.

StringBuilder, StringBuffer, and Thread Safety

`StringBuilder` is mutable and not synchronized. It is intended for local construction where one thread owns the builder.

`StringBuffer` provides synchronized operations. It exists largely for compatibility and specific shared-mutation requirements. Sharing a mutable buffer between threads is rarely the best design because a sequence of individually synchronized calls may still require a larger atomic operation.

```
String buildAuditLine(AuditEvent event) {
    return new StringBuilder(128)
        .append(event.timestamp())
        .append(' ')
        .append(event.type())
        .append(' ')
        .append(event.safeDescription())
        .toString();
}
```

The builder is local, so synchronization is unnecessary. Supplying an estimated capacity can reduce internal growth when size is predictable, but aggressive over-allocation wastes memory. Measure hot paths before adding capacity calculations everywhere.

Substrings, Splitting, and Hidden Allocation

Operations that produce many strings can dominate allocation even when each individual line looks harmless.

```
String[] fields = line.split(",");
```

`split()` interprets its argument as a regular expression. In a high-volume parser, regex processing, result-array allocation, and creation of field strings may be significant. A dedicated parser or index-based scan can be faster when profiling proves this path is hot.

Modern JDK String substring operations create an independent representation of the requested content rather than retaining a tiny view of a large original character array. This avoids accidentally retaining a very large source through a small substring, though substring-heavy processing still allocates results.

Streams of logs, CSV files, and protocol messages should be processed incrementally. Reading an entire large payload into one String and then producing thousands of derived strings increases peak memory and GC pressure.

Text Blocks and Readability

Text blocks provide a readable representation for multi-line strings.

```
String query = """
    select id, customer_number, total
    from orders
    where status = :status
    order by created_at desc
    """;
```

They improve source readability but do not make dynamic SQL safe. Parameters must still be bound through the database API. Text blocks also have indentation and line-ending rules that should be tested when exact external formatting matters.

Use them for embedded examples, templates, JSON, or SQL when the content genuinely benefits from preserving a multi-line shape. Do not use them to hide a large external resource that should be versioned and loaded separately.

Strings and Sensitive Data

String immutability is usually an advantage, but it complicates handling secrets. Once a password is placed in a String, application code cannot overwrite its internal contents. Copies may also be created by parsing, logging, concatenation, heap dumps, or framework conversion.

Some APIs accept `char[]` or `byte[]` so the caller can overwrite the array after use:

```
char[] password = readPassword();

try {
    authenticate(username, password);
} finally {
    Arrays.fill(password, '\0');
}
```

Clearing the array reduces exposure but is not a complete guarantee: libraries may have copied it, and runtime behavior can leave other representations. The broader controls remain important: minimize secret lifetime, avoid logs and exception messages, restrict heap-dump access, use secret managers, and prefer APIs designed for credentials.

Interning secrets or high-cardinality personal data is inappropriate because it deliberately introduces shared retention.

Production Perspective

String-related production problems usually arise from allocation, retention, encoding, or accidental disclosure rather than from the basic API.

A service may create millions of temporary strings while serializing responses or assembling logs. A cache may retain large request bodies through string values. A parser may load a multi-gigabyte file into memory before splitting it. An equality bug may use `==` and fail only when input comes from JSON rather than a literal. Unicode truncation may corrupt an emoji or customer name. A password may appear in a heap dump because it was converted to `String`.

Useful evidence includes:

- Allocation profiling from Java Flight Recorder or another profiler.
- Heap histograms showing `String` and backing-array counts.
- Dominator analysis identifying which cache, queue, or request object retains strings.
- GC logs showing allocation rate and live-set growth.
- Request-size and serialization metrics.
- Tests using dynamically constructed and non-ASCII input rather than literals only.

Avoid guessing from the number of `+` operators in source code. The compiler and JVM optimize many ordinary expressions. Profile representative workloads and fix the operation responsible for material allocation or retention.

Interview-Ready Summary

Java `String` is an immutable value type. Operations that appear to modify it produce a result value, while the original remains unchanged. Immutability enables safe sharing, stable hashing, concurrent reads, and `String` pooling.

`String` literals refer to canonical interned values. `new String("Java")` explicitly creates a distinct object, while the literal refers to the pooled representative. `==` compares references and may appear to work for literals; `equals()` must be used for content.

`intern()` returns the canonical pooled reference for an equal value. It does not modify the receiver. Interning can reduce duplication for a controlled low-cardinality vocabulary, but it is not a default optimization for arbitrary application input.

Compile-time constant concatenation can be folded and pooled. Runtime concatenation constructs a result using compiler and JVM strategies. `+` is appropriate for small readable expressions, while `StringBuilder` or a joining API avoids repeated immutable reconstruction in loops.

A `String` is a sequence of UTF-16 code units, so `length()` is not always the number of user-visible characters. Production design must also consider allocation, large-payload retention, locale and Unicode semantics, and the special handling required for secrets.

Revision Notes

- `String` is final and immutable; variables referring to strings can still be reassigned.
- Immutability enables safe sharing, stable hash keys, pooling, and unsynchronized reads.
- `String` literals use canonical pooled references.

- Canonical means the shared representative selected for an interned value.
- `new String(existing)` creates a distinct immutable object and is normally unnecessary.
- Use `equals()` for content and `==` only for intentional reference identity.
- `intern()` returns the pooled canonical reference; it does not modify the receiver.
- Do not intern arbitrary high-cardinality, untrusted, sensitive, or short-lived data by default.
- Compile-time constant concatenation may be folded and pooled.
- Use `+` for small clear expressions; use builders or joining APIs for repeated construction.
- `StringBuilder` is unsynchronized and suitable for thread-confined construction.
- `split()` uses a regular expression and can create substantial allocation in hot parsers.
- `length()` counts UTF-16 code units, not necessarily user-visible characters.
- Text blocks improve source formatting but do not provide SQL safety or external-resource management.
- Minimize the lifetime and copying of sensitive text; avoid representing secrets as interned Strings.
- Diagnose String performance with allocation and retention evidence, not source-level folklore.